



Universitat
Oberta
de Catalunya

(HHTMLCSS1) Herramientas HTML y CSS I
PEC 1 - Herramientas para desarrolladores de los navegadores y workflow front-end moderno

Marc Turu Roca

Profesor: Mariona Roca Ros

Fecha de entrega: 3/4/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

1. Introducción	2
2. Configuración inicial	2
2.1 Plantilla con Parcel	2
2.2 Entornos de desarrollo y producción	3
2.3 Soporte a navegadores antiguos	3
2.4 Pre/postprocesadores	4
2.5 Dependencias externas	6
3. Desarrollo	7
3.1 Idea y diseño	7
3.2 Estructuración de los archivos	8
3.3 Semántica y accesibilidad	9
3.4 Interactividad con JavaScript	10
4. Despliegue	11
4.1 Versionado	11
4.2 Publicación a internet	11
5. Recursos	12
6. Anexos	13

1. Introducción

Esta práctica ha consistido en el desarrollo de un sitio web aplicando conceptos de workflows modernos de desarrollo front-end y las herramientas para trabajar con ellos. Junto con los conceptos y actividades iniciales, se llevó a cabo todo este proceso: desde las instalaciones y configuración iniciales hasta el despliegue del resultado en internet, pasando por fases de estructuración, desarrollo y mejoras.

El repositorio de **GitHub** es: <https://github.com/marcturu/saboreamexico>

El link para acceder a la **web** es: <https://saboreamexico.netlify.app/>

2. Configuración inicial

2.1 Plantilla con Parcel

Para esta primera parte, a parte de actualizar la versión de Node para poder utilizar herramientas modernas, se creó el *package.json* inicial ejecutando `npm init -y`, y se eliminó el campo autogenerado *main* para que no interfiriera con la configuración de *Parcel*.

A continuación, se ejecutó `npm install --save-dev parcel` para instalar *Parcel* y se empezó con el *setup* del proyecto creando *src/index.html*, *src/styles.css* y *src/app.js*. Después de ejecutar `npm run start` para probar el servidor, se añadió el siguiente campo y *scripts* en el *package.json*, respectivamente¹:

```
"source": "src/index.html",
```

```
"start": "parcel",
```

```
"build": "parcel build",
```

De esta manera, al ejecutar `npm run build`, se podía comprobar cómo se creaba la carpeta *dist/* en la cual se ubicaba el código optimizado para producción.

Por otro lado, al ejecutar `npm run start`, se podía comprobar cómo se visualizaba la web (por ahora en un estado muy básico) en `http://localhost:1234`, en la cual se podían realizar cambios y visualizarlos en tiempo real en el navegador. De esta manera, el/la programador/a tiene la libertad de adaptar la web de manera más visual y óptima.

¹ Se profundiza más en la explicación de este campo y scripts en el apartado **2.2 Entornos de desarrollo y producción**

2.2 Entornos de desarrollo y producción

Antes de acabar de pulir las definiciones de los entornos de desarrollo y producción -ya que antes ya se declararon los scripts para ello- se ejecutó `npm install --save-dev rimraf npm-run-all` para añadir estas dependencias de desarrollo en el `package.json`². La primera (`rimraf`) con la función de eliminar carpetas de forma multiplataforma y la segunda (`npm-run-all`) para ejecutar múltiples *scripts* del `package.json` en secuencia con un solo comando.

De esta manera, y añadiendo/editando las siguientes líneas en el `package.json`:

```
"parcel:dev": "parcel",
"parcel:build": "parcel build",
"clean": "rimraf dist .parcel-cache",
"start": "npm-run-all clean parcel:dev",
"build": "npm-run-all clean parcel:build",
```

con `rimraf` se eliminaban completamente los archivos presentes dentro de `dist/` antes de cada `build` y se generaban desde cero. A diferencia de antes, así se podía asegurar la eliminación de archivos obsoletos ocupando espacio, solucionar posibles confusiones con versiones de archivos, tener un despliegue más limpio y predecible y evitar *bugs* con versiones antiguas de caché.

A continuación se explica individualmente cada *script* hasta este punto³:

- **parcel:dev**: Lanza parcel en modo desarrollo con live reload
- **parcel:build**: Compila y optimiza para producción
- **clean**: Elimina `dist/` y `.parcel-cache`
- **start**: Primero limpia y luego lanza el servidor de desarrollo⁴
- **build**: Primero limpia y luego compila para producción

2.3 Soporte a navegadores antiguos

Como se menciona en el enunciado: “Configura el proyecto para que la versión de producción sea compatible con navegadores antiguos, con versiones activas desde hace 10 años.” se optó por usar la siguiente configuración⁵ en el `package.json`:

```
"browserslist": "last 10 years",
```

² Estas se añadieron en **devDependencies** porque solamente eran necesarias para desarrollo.

³ Más adelante se añadió **stylelint** y se actualizaron.

⁴ En ocasiones también se usa **dev** (`npm run dev`).

⁵ Extraída del siguiente recurso: <https://github.com/browserslist/browserslist#full-list>.

De esta forma, se seleccionaban todos los navegadores con versiones lanzadas desde hace exactamente 10 años hasta hoy, lo que incluía navegadores antiguos y versiones viejas de *Safari*, *Chrome* y *Firefox*, obligando a *Parcel* (con sus herramientas por defecto como *Autoprefixer*, por ejemplo) a transformar el código moderno a sintaxis más antigua compatible con todos ellos.

Se puede comprobar qué incluye ejecutando: `npx browserslist "since 2015"`.

2.4 Pre/postprocesadores

Continuando con la línea de configuración seguida hasta el momento, tras ejecutar `npm run build` se pudo observar que el CSS generado en la *dist/* aparecía minificado, sin saltos de línea y con optimización de propiedades, como la eliminación de *bottom* en *linear-gradient*, ya que es el valor por defecto. Estos cambios fueron resultado del proceso de optimización del bundler (*Parcel*) junto con sus postprocesadores **que ya vienen por defecto**. Además, aunque *Autoprefixer* estaba activo, no se añadieron prefijos como *-webkit-linear-gradient* porque la configuración previa de *browserslist* incluía navegadores que ya soportaban esta propiedad sin necesidad de añadir nada. En cambio, sí que la añadió con *-webkit-backdrop-filter*, por ejemplo.

Seguidamente, se ejecutó `npm i -D posthtml-include` para instalar **posthtml-include**, un plugin que permitiría incluir código *HTML* desde un fichero externo dentro de otro fichero *HTML*. Esto resultaría útil para construir elementos que se repetirían dentro de un mismo sitio web estático, como una cabecera o pie de página, y para dividir el código y mejorar su legibilidad.

Una vez instalado, se creó el fichero *.posthtmlrc* en la raíz del proyecto y se incluyó el siguiente comando de configuración:

```
{ "plugins": { "posthtml-include": {} } }
```

Aparte, se decidió usar una metodología CSS junto con dos preprocesador para poder trabajar de manera más ordenada:

En primer lugar, se instaló **SASS** `npm install --save-dev sass` para que *Parcel* compilara automáticamente los archivos *.scss* que usaría a *CSS*. Junto con este, se organizó la estructura del código (sobretudo los estilos) para usar la metodología **ITCSS** (para estructurar las carpetas y jerarquía de capas en *SCSS* en *src/assets/styles*), **BEM** (en *components/* para usar nombres de clases consistentes

y predecibles) y **OOCSS** (en *objects/* para definir bloques reutilizables con su propia estructura). El hecho de trabajar con estos recursos, resultaría muy útil para empezar un desarrollo con buenas bases y continuarlo de manera escalable.

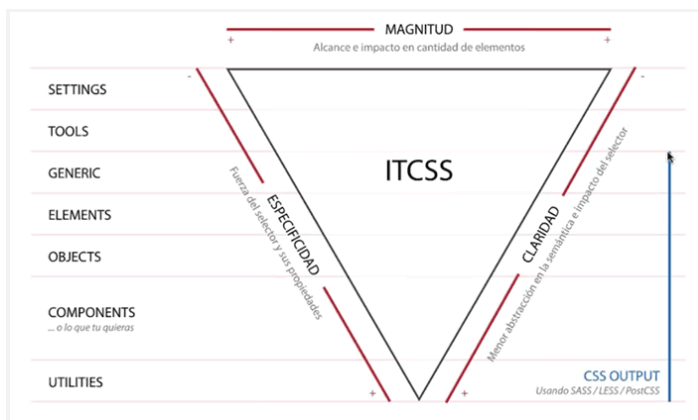


Fig. 1 - Representación visual de la metodología ITCSS

En segundo lugar, se instaló el linter **Stylelint** `npm install --save-dev stylelint@^17.5.0 stylelint-config-recommended-scss@^17.0.0 stylelint-config-prettier-scss@1.0.0 stylelint-scss@^7.0.0` para analizar el código automáticamente y detectar errores, malas prácticas e inconsistencias de estilo. Esto se podía hacer a partir de las reglas especificadas en el archivo `.stylelintrc`, como por ejemplo la siguiente expresión para evaluar el nombre de las clases:

```
"selector-class-pattern":
"^([a-z])([a-z0-9-]+)?(__([a-z0-9-]+-?)+)?(--([a-z0-9-]+-?)+){0,2}$"
```

o esta otra para evitar usar valores *HEX* inválidos:

```
"color-no-invalid-hex": true,
```

Además de las funcionalidades ya incorporadas en las extensiones y *plugins* instalados, en los cuales se comprobaron sus compatibilidades haciendo:

```
Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-scss peerDependencies
{ stylelint: '^16.8.2 || ^17.0.0' }

Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-config-recommended-scss peerDependencies
{ postcss: '^8.3.3', stylelint: '^17.0.0' }

Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-config-prettier-scss peerDependencies
{ stylelint: '>=15.0.0' }
```

Fig. 2 - `npm info <dependencia> peerDependencies` para cada dependencia a instalar

Estas representaban (por orden en el comando):

- El núcleo del linter CSS
- Reglas recomendadas para SCSS
- Evita conflictos con *Prettier*.
- Plugin para reglas SCSS específicas (variables, nesting, mixins).

Finalmente, se añadió el siguiente comando para que, antes de hacer la *build* para producción, se verificara el cumplimiento de las reglas de *stylelint*.

```
"build": "npm-run-all stylelint clean parcel:build"
```

2.5 Dependencias externas

Como dependencias externas, se incorporaron:

- **AOS** (^2.3.4). Una librería ligera de JavaScript que permite implementar animaciones activadas por el desplazamiento (*scroll*) del usuario.

La principal razón de su adición fue mejorar la experiencia visual que ofrecía la página web mediante efectos dinámicos sin necesidad de desarrollar lógica compleja desde cero y, en ocasiones, menos optimizada.

- **@fortawesome/fontawesome-free** (^7.2.0). Una biblioteca ampliamente utilizada que proporciona una gran colección de iconos vectoriales escalables. La principal razón de su adición fue mejorar la integración de elementos visuales en la interfaz, mejorando la usabilidad y la estética del proyecto sin impacto significativo en el rendimiento.

3. Desarrollo

3.1 Idea y diseño

Debido a que el proyecto consistía en desarrollar una página web en torno a la temática **gastronomía**, se propuso hacerlo sobre la **mexicana**, ya que la gran variedad de sabores y alimentos que presenta se traducía a una gran variedad de oportunidades y diseños a la hora de implementar la página web.

De esta idea nació:



Fig. 3 - Logotipo de Saborea México

Gracias al uso de *PostHTML*, la página web se estructuró a través de las inclusiones de diferentes componentes:

0. **Header y Footer.** Logo y barra de navegación, y créditos de la página.
1. **Home** (index.html)
 - a. **Hero.** Imagen principal con un efecto de sombreado y difuminación acompañada del texto de portada y botón de navegación.
 - b. **Intro.** Párrafo introductorio.
 - c. **Featured.** Presentación de dos recetas destacadas (det1 y det2) con un botón para ver más.
 - d. **Regions.** Información extra para cerrar la página.
2. **Categoría** (category.html)
 - a. **Hero.** Título y subtítulo de la página.
 - b. **Recipe list.** Todas las recetas en forma de grid.
3. **Detalle 1** (det1.html)
 - a. **Hero.** Título acompañado con una imagen general.
 - b. **Recipe detail.** Conjunto de 3 imágenes con 3 párrafos, un vídeo y botones simples de navegación entre páginas.

4. **Detalle 2** (det2.html)
 - a. **Hero**. Título acompañado con una imagen general.
 - b. **Recipe detail**. Conjunto de 3 imágenes con 3 párrafos, un vídeo y botones simples de navegación entre páginas.
5. **Fuentes** (links.html)

Se optó por seguir un estilo de diseño **mobile-first**, es decir, programando primero la versión para dispositivos móviles y adaptándola a resoluciones mayores.

Para ir revisando la implementación, aparte de las técnicas comentadas en los módulos anteriores (sobretudo en el Módulo 1), se hizo uso de [Response Viewer](#), una extensión de chrome que permite ver a tiempo real la página web para diferentes resoluciones (con posibilidad de personalización).

3.2 Estructuración de los archivos

```
src/
├─ index.html
├─ categoria.html
├─ det1.html
├─ det2.html
├─ links.html
├─ views/                                # Parciales PostHTML
│  ├─ partials/
│  ├─ home/
│  ├─ categoria/
│  ├─ det1/
│  ├─ det2/
│  └─ links/
├─ assets/
│  ├─ fonts/                            # Fuentes autoalojadas (Unbounded, DM Sans)
│  ├─ images/
│  ├─ scripts/
│  │  ├─ app.js
│  │  └─ modules/                        # Módulos JS (aos y header)
│  └─ styles/
│     ├─ main.scss
│     ├─ _dependencies.scss
│     ├─ settings/                       # Variables, fuentes
│     ├─ tools/                          # Funciones, mixins
│     ├─ generic/                        # Reset
│     ├─ elements/                       # Estilos base
│     ├─ objects/                        # Objetos OOCSS
│     ├─ components/                     # Componentes BEM
│     └─ utilities/                      # Clases de utilidad
```

Fig. 4 - Estructura de los archivos del proyecto (disponible en el [README.md](#))

Se separaron:

- Las diferentes páginas del **src/** (index.html,...)
- Las diferentes secciones de **views/** (partials/...)
- Los diferentes recursos de los **assets/** (fonts,...)
- Las diferentes imágenes de las **images/** (recipes,...)
- Los diferentes scripts de los **modules/** (aos.js,...)
- Los diferentes estilos de los ITCSS **styles/** (settings/,...)

De esta manera, los archivos principales como *index.html*, *app.js* y *main.scss* (entre otros), tenían el código justo y necesario para poder importar y usar los demás componentes y funcionalidades, respectivamente.

También se optó por mejorar la resolución de las imágenes con [Squoosh](#).

3.3 Semántica y accesibilidad

Desde el inicio del desarrollo de la página web hasta su publicación final, se ha construido utilizando etiquetas semánticas de HTML5, lo que mejora la comprensión del contenido tanto para navegadores como para tecnologías activas.

Gracias también al uso de la metodología ITCSS, se ha podido mantener una jerarquía correcta, no solamente de código y valores, sino de carpetas y archivos.

El desarrollo ha seguido las pautas de accesibilidad de [W3C](#).

A continuaciones se comentan las más relevantes:

- WCAG 1.1.1 Contenido no textual

aria-hidden="true" en iconos decorativos.

- WCAG 1.4.1 Uso del color

aria-current="page" para indicar la página activa (no solo color sino también con subrayado)

- WCAG 2.4.1 Evitar bloques

```
<a href="#main-content" class="header__skip-link">
```

- WCAG 2.4.7 Foco visible

```
&:focus-visible
```

- WCAG 4.1.2 Nombre, función, valor

- *aria-label* en todos los *<nav>* | *aria-expanded* y *aria-controls* en el botón hamburguesa | *arialabelledby="titulo-de-la-seccion"*

- La relación texto/fondo cumple ratio mínimo AA.

3.4 Interactividad con JavaScript

El uso de JavaScript en la aplicación se ha dividido en dos secciones:

- **AOS.** En el archivo *scripts/aos.js* se importaba la dependencia y se inicializaba con valores como

```
AOS.init({
  duration: 550,
  easing: 'ease-out',
  once: false,
  offset: 60,
});
```

- **Header.** En el archivo *scripts/header.js* se llevaban a cabo más funcionalidades, sobre todo con la interacción con elementos del DOM para controlar el comportamiento del header dependiendo de resoluciones y páginas.

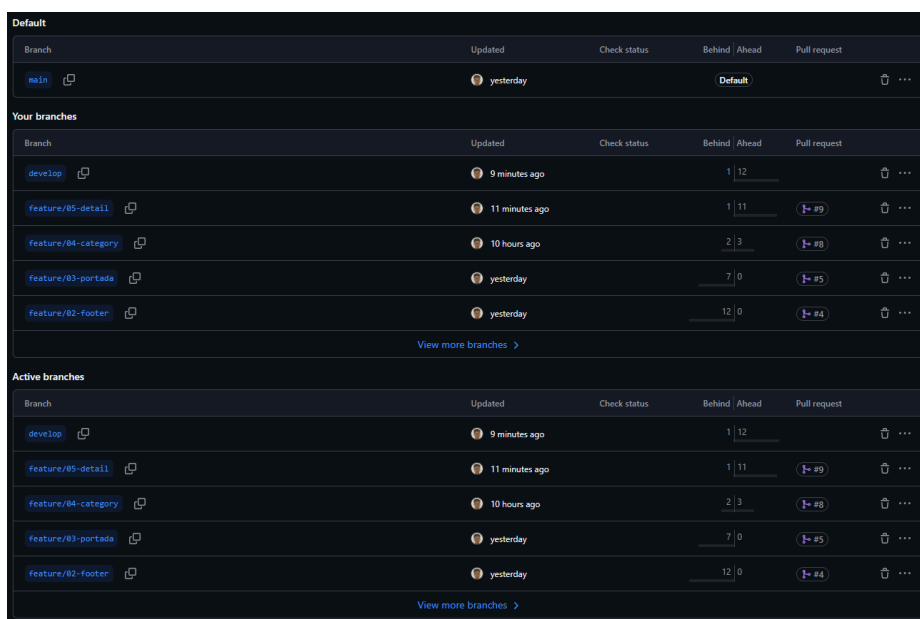
Con **querySelectors** se obtenían los elementos, y había funciones destacadas como **updateHeaderOpacity** para actualizar la opacidad del header dependiendo de *Math.min(window.scrollY / 1000, 1)*, **menuBtn.addEventListener('click', () {})** para abrir y cerrar el menú hamburguesa de móvil, **link.addEventListener('click', () {})** para cerrar el menú móvil al hacer *click* en un enlace y **logo.addEventListener('click', () {})** para cerrar el menú móvil al hacer *click* en el logo.

4. Despliegue

4.1 Versionado

Debido a que ya tenía una cuenta de GitHub, creé un nuevo [repositorio](#) y lo enlacé con la carpeta donde almacenaba el código y en la que había realizado las instalaciones y configuraciones iniciales.

GitHub permitió tener un mejor control sobre las versiones, ya que se mantuvo la rama **main** para presentar los “lanzamientos” de las versiones finales y se creó una rama **develop** para poder juntar las diferentes **features/xx-nombre-feature** que se iban desarrollando a lo largo del proyecto.



Branch	Updated	Check status	Behind	Ahead	Pull request
main	yesterday			(Default)	
Your branches					
develop	9 minutes ago		1	12	
feature/05-detail	11 minutes ago		1	11	#9
feature/04-category	10 hours ago		2	3	#8
feature/03-portada	yesterday		7	0	#5
feature/02-footer	yesterday		12	0	#4
View more branches >					
Active branches					
develop	9 minutes ago		1	12	
feature/05-detail	11 minutes ago		1	11	#9
feature/04-category	10 hours ago		2	3	#8
feature/03-portada	yesterday		7	0	#5
feature/02-footer	yesterday		12	0	#4
View more branches >					

Fig. 5 - Ramas de GitHub

4.2 Publicación a internet

Finalmente, con la creación del entorno de despliegue en *Netlify* junto con la configuración de la cuenta de *GitHub* con el repositorio, la rama adecuada de despliegue, la edición de los parámetros necesarios, y la carpeta de producción para poder acceder a la [URL](#), se pudo visualizar la web en internet a la vez que apreciar el *continuous deployment* que ofrecía la plataforma cuando se actualizaba la rama de despliegue con nuevos cambios.

5. Recursos

- [Gastronomía mexicana — UNESCO](#)
Información sobre el reconocimiento de la gastronomía mexicana como Patrimonio Cultural de la Humanidad (2010).
- [Historia de los Tacos al Pastor](#)
Origen e historia de los tacos al pastor y su relación con la inmigración libanesa en México.
- [Historia de los Nachos Tradicionales](#)
Origen e historia de los nachos y su información más destacada como ingredientes y valores nutritivos.
- [Unsplash](#)
Imágenes de *hero*, recetas y galería obtenidas de Unsplash bajo licencia gratuita para uso comercial y personal.
- [Squoosh](#)
Compresor y optimizador de imágenes directamente desde el navegador y con variedad de formatos.
- [Tacos al Pastor — YouTube](#)
Vídeo de receta incrustado en la página de detalle. Autor: Animal Gourmet.
- [Nachos Tradicionales — YouTube](#)
Vídeo de receta incrustado en la página de detalle. Autor: Cuk-it!.
- [Parcel](#)
Module bundler utilizado para el desarrollo y build de producción.
- [Sass / SCSS](#)
Preprocesador CSS utilizado para los estilos del proyecto.
- [AOS — Animate On Scroll](#)
Librería utilizada para las animaciones al hacer scroll.
- [Font Awesome](#)
Librería de iconos utilizada en todo el proyecto.

6. Anexos

1. Aplicación web. Saborea México (Netlify)

<https://saboreamexico.netlify.app/>

2. Capturas de pantalla (Drive):

https://drive.google.com/drive/folders/17R6RNTIWdgnpdvqA-tYFQzjtBnVTleSB?usp=drive_link

3. Repositorio con el código (GitHub):

<https://github.com/marcturu/saboreamexico>

Licencia Unsplash:

Licencia

Las imágenes de Unsplash se utilizan libremente. Nuestra licencia lo refleja.

- ✓ Todas las imágenes se pueden **descargar** y utilizar de **forma gratuita**
- ✓ **Fines comerciales y no comerciales**
- ✓ **No se necesita permiso** (¡aunque se agradece la atribución!)

Qué no está permitido 🚫

- ⊘ Las imágenes no se pueden **vender** sin modificaciones significativas.
- ⊘ La recopilación de imágenes de Unsplash para replicar un servicio similar o contrapuesto.

Archivos

Unsplash te concede una licencia de derechos de autor irrevocable, no exclusiva y mundial para descargar, copiar, modificar, distribuir, realizar y utilizar imágenes de Unsplash de forma gratuita, incluso con fines comerciales, sin el permiso o la atribución del fotógrafo o Unsplash. Esta licencia no incluye el derecho a compilar imágenes de Unsplash para replicar un servicio similar o contrapuesto.